

Python Questions & Answers

Knowledge Base Document for a RAG (Retrieval-Augmented Generation) System

Python Basics

Q: What is Python?

A: Python is a high-level, interpreted, general-purpose programming language known for its readable syntax and dynamic typing. It is widely used in web development, data science, automation, and AI/ML.

Q: How do you declare a variable in Python?

A: Python does not require any keyword to declare a variable. You simply assign a value using `name = value`, and Python automatically infers the type (dynamic typing).

```
x = 10
name = "Claude"
pi = 3.14
```

Q: What are the built-in data types in Python?

A: The main built-in data types are: `int`, `float`, `str`, `bool`, `list`, `tuple`, `dict`, `set`, and `NoneType`. There are also specialized types like `bytes` and `frozenset`.

Q: What is the difference between a list and a tuple?

A: A list is mutable (its contents can be changed), while a tuple is immutable. Lists use square brackets `[]` and tuples use round brackets `()`. Tuples are generally faster and more memory-efficient.

```
my_list = [1, 2, 3]
my_tuple = (1, 2, 3)
my_list.append(4) # OK
# my_tuple.append(4) # Error
```

Q: Why does indentation matter in Python?

A: Python uses indentation (spaces/tabs) instead of curly braces `{}` to define code blocks like loops, functions, and conditionals. Incorrect indentation raises an `IndentationError`.

Functions & OOP

Q: How do you define a function in Python?

A: Functions are defined using the `def` keyword. A function can take parameters and return a value using the `return` statement.

```
def add(a, b):
    return a + b

result = add(3, 5)
```

Q: What are *args and **kwargs?

A: *args allows a function to accept a variable number of positional arguments (as a tuple), while **kwargs allows a variable number of keyword arguments (as a dictionary).

```
def demo(*args, **kwargs):  
    print(args)  
    print(kwargs)  
  
demo(1, 2, name="A", age=30)
```

Q: What is a lambda function?

A: A lambda is an anonymous, single-expression function defined using the lambda keyword. It is typically used for small, one-line functions, often with map, filter, or sorted.

```
square = lambda x: x * x  
print(square(5)) # 25
```

Q: What are classes and objects?

A: A class is a blueprint that defines attributes and methods. An object is an instance of a class. Classes are defined using the class keyword.

```
class Dog:  
    def __init__(self, name):  
        self.name = name  
    def bark(self):  
        return f"{self.name} says Woof!"  
  
d = Dog("Rex")  
print(d.bark())
```

Q: What is inheritance?

A: Inheritance is an OOP concept where a child class inherits attributes and methods from a parent class, enabling code reuse.

```
class Animal:  
    def speak(self):  
        return "..."  
  
class Cat(Animal):  
    def speak(self):  
        return "Meow"
```

Q: What does the __init__ method do?

A: __init__ is the constructor method that is called automatically when an object is created. It is used to set the object's initial attributes.

Data Structures & Control Flow

Q: What is a dictionary?

A: A dictionary is an unordered (insertion-ordered since Python 3.7+) collection of key-value pairs, written with curly braces {}. Values can be quickly accessed by their key.

```
student = {"name": "Riya", "age": 21}
print(student["name"])
```

Q: What is a list comprehension?

A: A list comprehension is a compact way to create a list, combining a loop and condition into a single line.

```
squares = [x**2 for x in range(10) if x % 2 == 0]
```

Q: What is the difference between a for loop and a while loop?

A: A for loop iterates over an iterable (list, tuple, range, etc.), while a while loop repeats as long as a condition remains true. A for loop's end is usually predetermined; a while loop's may not be.

```
for i in range(5):
    print(i)

n = 0
while n < 5:
    print(n)
    n += 1
```

Q: What is try-except used for?

A: The try-except block is used to handle errors/exceptions so the program can fail gracefully instead of crashing.

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
```

Q: What is a set and when should you use it?

A: A set is an unordered collection of unique elements. It's used to remove duplicates or speed up membership testing.

```
nums = [1, 2, 2, 3, 3, 3]
unique = set(nums) # {1, 2, 3}
```

Modules, Files & Advanced Topics

Q: What are modules and packages in Python?

A: A module is a single .py file containing functions/classes. A package is a collection of modules, organized as a directory containing an __init__.py file.

```
import math
from mypackage import mymodule
```

Q: How is file handling done in Python?

A: Files are opened using the `open()` function, and using a `with` statement ensures the file is automatically closed.

```
with open("data.txt", "r") as f:
    content = f.read()
```

Q: What is a decorator?

A: A decorator is a function that wraps another function to modify its behavior without changing its code. It is applied using the `@` symbol.

```
def my_decorator(func):
    def wrapper():
        print("Before")
        func()
        print("After")
    return wrapper

@my_decorator
def say_hello():
    print("Hello")
```

Q: What is a generator?

A: A generator is a special function that returns values using the `yield` keyword, without loading everything into memory at once. It's memory-efficient for large datasets.

```
def count_up_to(n):
    i = 1
    while i <= n:
        yield i
        i += 1
```

Q: What is the GIL (Global Interpreter Lock)?

A: The GIL is a mechanism in the Python interpreter that allows only one thread to execute Python bytecode at a time. This limits the benefit of multi-threading for CPU-bound tasks, though multiprocessing can work around it.

Q: What is pip?

A: pip is Python's package manager, used to install, uninstall, and manage third-party libraries from PyPI (the Python Package Index).

```
pip install requests
pip uninstall requests
pip list
```

Q: Why use a virtual environment?

A: A virtual environment creates an isolated Python environment for each project, so dependencies across different projects don't conflict.

```
python -m venv myenv
source myenv/bin/activate # Linux/Mac
myenv\Scripts\activate    # Windows
```